Lesson 8 · A Stateful Agent with LangGraph

PDF: this lesson · **Install (Linux · macOS · Windows):** guide

> **Part of local-ai-lab** - a hands-on course for building local AI.

> **Interactive version (slides):** https://nikolareljin.github.io/local-ai-lab/lesson-8-langgraph.html **Read it locally (no GitHub Pages):** `./run -l 8 lesson` **Course home:** https://nikolareljin.github.io/local-ai-lab/ **Source:** https://github.com/nikolareljin/local-ai-lab **Author:** Nik Reljin **Time:** ~60-75 min · **Prerequisites:** Lessons 1 and 7 (Lesson 2 helpful) · full objectives in SYLLABUS.md

> **Lessons:** 1 · RAG → 2 · MCP → 3 · Hybrid retrieval → 4 · RAG safety → 5 · RAG evaluation → 6 · Repo assistant → 7 · LangChain → **8 · LangGraph (you are here)** → 9 · Ollama tools → 10 · Semantic Kernel → 11 · Bedrock Agents → 12 · Google ADK → ... → 15 · Docs from changes

> **Status: working demo.** Runnable in **Python and Node.js**. The second lesson that is **not** dependency-free - and, like Lesson 7, the dependency is part of the argument.

First - what is LangGraph?

If you have not used it: **LangGraph is a small state-machine library with checkpoints.** It is not LangChain 2.0, and it is not an agent framework in the "hand it some tools and hope" sense. Three things are worth knowing before you read a line of it:

1. **You declare a typed state**, write nodes that return **updates** to that state, and connect them with edges - including edges that point backwards. That is the whole model.
2. **It persists.** Every step can be checkpointed against a `thread_id`, which is what lets a run survive the process, and what lets a run stop halfway and be resumed later by someone else.
3. **It depends on `langchain-core`.** So this lesson sits **on top of** Lesson 7's dependency rather than beside it, and what it costs you depends entirely on whether you were already paying that.

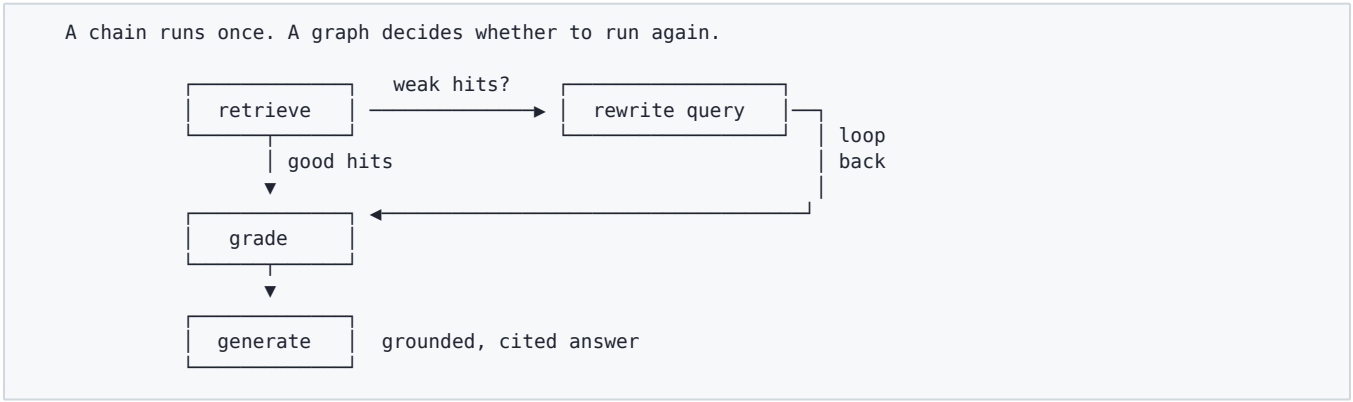
It ships official Python and JavaScript SDKs. Both are here.

> **Why this lesson, right after LangChain?** Lesson 7 asked what a framework bought and what it cost. This is the same question one level up, and it has a harder version: not "is a loop better than a chain" - it obviously is - but **"is LangGraph better than a `while` loop".**

What you'll learn

In Lesson 1 you built a pipeline that retrieves once and answers. Lesson 7 rebuilt it on LangChain and it still retrieved once and answered. Both are **linear**: whatever the first search returns is what the model gets, and if that search was bad, the answer is bad.

Real assistants do not work like that. They notice weak evidence and search again. They stop and ask before doing something irreversible. They remember the last thing you said.



- **State** - a typed struct that flows between nodes, and the difference between a field that **accumulates** and one that **replaces**
- **Nodes and edges** - retrieval, grading, rewriting and generation as separate, testable steps
- **Conditional edges and cycles** - branching on "is this evidence good enough?", and looping back to try a better query instead of answering badly
- **Two caps** - yours and the framework's, and why confusing them produces either a hang or a lie
- **Checkpoints and** `thread_id` - state that outlives the process, and a run you can resume
- `interrupt()` - stopping before an irreversible answer and handing a human the evidence
- **Observability** - and the fact that LangChain's hosted tracing client is already installed on your machine, whether you asked for it or not

> **The one idea:** a chain runs once; a graph decides whether to run again - and can stop halfway and ask you. Everything else in this lesson is about what that costs.

The demo

The corpus is **Lesson 7's**, referenced by path rather than copied:

```
CORPUS_DIR = ROOT / "lessons" / "07-langchain-rag" / "data" / "corpus"
```

Seven markdown documents for a fictional Aurora X1 field sensor - manual, FAQ, API, troubleshooting, networking, warranty, installation. The strongest form of "the same corpus" is the same bytes, and a copy would drift the first time either lesson was edited. Nothing was added or reworded to make the graph look good; a test asserts the path still resolves under `07-langchain-rag`.

Three arms, not two:

Arm	What it is	Answers the question
linear	retrieve once, cite, answer. Lesson 1 and Lesson 7's shape	**Why loop at all?**

loop	the corrective loop as 61 lines of plain `while`	**Why LangGraph and not a `while` loop?**
graph	the same loop as a `StateGraph`, with a checkpointer and an interrupt	-

The middle arm is the important one. Benchmarking LangGraph against a single-shot chain would prove that **looping** helps, which is not the same claim, and a reader would be entirely right to answer "so write a while loop". Lesson 7's authority came from refusing to benchmark against a straw man. Dropping that standard one lesson later would be worse than never having had it.

Nine questions, and each one is in `data/questions.json` with a stated reason:

#	Question	Should reach	Why it is here
q1	What is the factory reset procedure?	`manual.md`	control, carried from Lesson 7
q2	Why does the status ring stay amber?	`faq.md`	control
q3	Which endpoint exports the logging buffer?	`api.md`	control
q4	The light is **orange** and it will not connect.	`faq.md`	the docs say **amber status ring**
q5	How do I **wipe** the device and start over?	`manual.md`	the docs say **factory reset**
q6	What paperwork does an **RMA** need?	`warranty.md`	the docs say **warranty claim**
q7	Is **5GHz** supported?	`networking.md`	the docs spell out **five gigahertz**
q8	Can I mount it sideways?	`installation.md`	**already right** - the loop pays for nothing
q9	What is the MTBF?	**nothing**	not in the corpus; it must give up, not spin

Those four failures are not subtle and they are not rigged. They are lexical: the words the reader used do not appear in any document, so no amount of ranking will find the right one. This is the single most common way retrieval fails in a real system, and it is invisible until somebody reads the citations.

> **Why compare retrieval and control flow, not the generated answer?** Prose from a model is different every run and cannot be committed to a file and diffed. Which documents were retrieved, how many attempts it took, and which branch was taken are all exactly reproducible. So the demo is scored on **which document is cited first**, or on whether the

> arm correctly refused - never on how the answer reads.

Run it

```
./run -l 8 demo                   # the three arms and the scorecard
./run -l 8 --lang node demo       # the same control flow in Node.js
./run -l 8 trace "Is 5GHz supported?"
```

```
      linear  loop    graph   att  question
 ----------------------------------------
 q1   PASS    PASS    PASS    1    What is the factory reset procedure?
 q2   PASS    PASS    PASS    1    Why does the status ring stay amber?
 q3   PASS    PASS    PASS    1    Which endpoint exports the logging buffer?
 q4   FAIL    PASS    PASS    2    The light is orange and it will not connect.
 q5   FAIL    PASS    PASS    2    How do I wipe the device and start over?
 q6   FAIL    PASS    PASS    2    What paperwork does an RMA need?
 q7   FAIL    PASS    PASS    2    Is 5GHz supported?
 q8   PASS    PASS    PASS    2    Can I mount it sideways?
 q9   FAIL    PASS    PASS    2    What is the MTBF?

The scorecard
                                      linear (L7)      loop (while) graph (LangGraph)
 top source correct                          4/8               8/8               8/8
 correctly abstained                         0/1               1/1               1/1
 ------------------------------------------------------------------------------------
 retrieval calls                               9        15  (+67%)                15
 grade calls                                   0                15                15
 rewrite calls                                 0                 7                 7
 ------------------------------------------------------------------------------------
 state survives a process                      -                 -      checkpointer
 pause / resume mid-run                        -                 -       interrupt()
 topology you can query                        -                 -       get_graph()
```

Read the top half first. **The chain gets 4 of 8 right and never refuses; the loop gets 8 of 8 and correctly refuses the ninth, for 67% more retrieval calls.** `q8` is in there deliberately: it was already correct on the first try, the grader panicked anyway, and one of those extra retrievals bought nothing at all. Leaving it out would have made the loop look free, and it is not.

There is no question here where the loop makes things **worse**. Thirteen candidates were tried and none regressed, which is worth saying plainly rather than manufacturing one for balance.

Now the bottom half, which is the actual finding:

```
  The while loop and the graph agree on all 9 questions: exactly.
```

LangGraph did not make the agent smarter. Same documents, same attempt counts, same retrieval counts, on every question - and a test asserts it, because if those two arms ever diverge the whole scorecard is void. Everything the framework cost was spent on the three rows the `while` loop leaves blank.

Experiment in the playground (needs Flask)

```
./run -l 8
```

Control	What moves

Top k	how much evidence reaches the grader
Grade threshold	at `0` nothing is ever weak, so the graph never loops - it **is** Lesson 7's chain. At `1.0` everything loops to the cap and abstains
Max attempts	your cap. At `1` the cycle is gone, by a different route
Rewrite the query	off = re-search the same words. A cycle that cannot change its own input is not a cycle
Human review	`0 pause · 1 approve · 2 veto` - the whole interrupt story on one slider
Show the checkpoint	`thread_id`, where the graph is standing, the counters, the full trace

Three things worth doing, in order. Ask **"The light is orange and it will not connect."** and read the **Every attempt** table - two rows, the first red. Then drag **Grade threshold** to 0 and watch the second row vanish: you have just turned the graph back into Lesson 7. Then set it back, leave **Human review** on `pause`, and move it to `approve` - watch **retrievals when paused** and **retrievals after resume** stay the same number.

Concept 1 · State, and what a reducer actually is

```
class AgentState(TypedDict, total=False):
   question: str          # the human's words. Never mutated.
   query: str             # the CURRENT search query. `rewrite` replaces this.
   attempt: int
   docs: List[Chunk]
   sources: List[str]
   grade: Grade

   trace: Annotated[List[str], operator.add]
   turns: Annotated[List[dict], operator.add]
   retrievals: Annotated[int, operator.add]
   grades: Annotated[int, operator.add]
   rewrites: Annotated[int, operator.add]
```

Two kinds of field, deliberately side by side. The bottom five are `Annotated[..., operator.add]`, so a node returning `{"retrievals": 1}` **adds one** to the running total. Everything above is last-write-wins, so a node returning `{"query": "..."}` **replaces** it.

That is what a reducer is, and it is much easier to see in one struct than to explain. It also has a practical payoff here: because the counters accumulate themselves, the scorecard's retrieval and rewrite numbers are **measured by the graph**, not asserted by the person writing the lesson.

Note what is **not** in that struct: the expected answer. `expect` lives in `data/questions.json` and is used only to score a finished run. A grader that can see the label is not grading.

Concept 2 · Conditional edges, and the edge that points backwards

```
builder.add_edge(START, "retrieve")
builder.add_edge("retrieve", "grade")
builder.add_conditional_edges("grade", route_after_grade,
       {"rewrite": "rewrite", "review": "review",
        "generate": "generate", "abstain": "abstain"})
builder.add_edge("rewrite", "retrieve")          # <- the cycle
```

`add_conditional_edges` takes a function returning a **string key** and a map from keys to nodes. The routing function never names a node directly:

```
def route_after_grade(state) -> str:
   if state["grade"]["verdict"] == "grounded":
       return "review" if HUMAN_REVIEW else "generate"
   if state["attempt"] >= MAX_ATTEMPTS:
       return "abstain"
   return "rewrite"
```

Because of that indirection the topology stays separable from the logic, which is what makes this possible:

```
  edges
   grade      -> abstain
   grade      -> generate
   grade      -> review
   grade      -> rewrite
   retrieve   -> grade
   review     -> generate
   review     -> retrieve
   review     -> veto
   rewrite    -> abstain
   rewrite    -> retrieve   <- the cycle
```

That is `graph.get_graph().edges`, printed by `./run -l 8 graph`. **A `while` loop cannot answer the question "what are your edges?" at runtime.** Its control flow is `if` and `while` - readable, but not queryable. Here it is a value. That sounds like a small thing until you are three months into a system nobody has drawn a diagram of.

Concept 3 · Two caps, and why you need both

Everyone adds the first one. Almost nobody understands the second until it fires.

Cap	Whose	What hitting it means
`max_attempts`	**yours**, in `data/questions.json`	domain logic. Routes to `abstain` and produces a **good** answer: "not in your documents"
`recursion_limit`	**LangGraph's**, in the invoke config	a structural floor. Raises `GraphRecursionError`

They are not two settings for the same thing. `max_attempts` says **how hard should this agent try**. `recursion_limit` says **this graph is misrouting and must be stopped** - it exists to catch a cycle whose **routing** is wrong, not one whose **search** is failing. Watch it fire:

```
./run -l 8 trace "What is the MTBF?" --recursion-limit 4
```

```
 GraphRecursionError after 4 steps - the framework's floor, not your ceiling.
Your max_attempts=3 would have abstained cleanly.
An agent without a domain cap is an infinite loop with an invoice.
```

Set only the framework's cap and a failing search reaches your users as a stack trace. Set only yours and a routing bug runs until something else stops it. The test suite asserts the unhappy path raises rather than hangs, which is worth more than any number of tests that the happy path works.

There is a third stop in this lesson, and it is the cheapest of the three: if the rewriter hands back a query it has already tried, the loop gives up immediately rather than spending another identical retrieval. **A cycle that cannot change its own input is an infinite loop with extra steps.**

Concept 4 · Checkpoints and `thread_id`

A checkpointer writes the state after every step, keyed by a `thread_id` you choose:

```
graph = builder.compile(checkpointer=MemorySaver())
config = {"configurable": {"thread_id": "aurora"}}
```

Everything that makes a graph different from a loop follows from that one line:

```
Memory - two turns on one thread
 thread 'aurora'  turn 1  What is the factory reset procedure?
 thread 'aurora'  turn 2  Which endpoint exports the logging buffer?
 thread 'somebody-else'  turn 1  <- a different thread_id remembers nothing
 checkpoints written on 'aurora': 10
```

`graph.get_state(config)` returns a snapshot with `.values`, `.config`, and - the useful one - `.next`, which for a paused graph is a tuple naming where it is standing. `get_state_history()` walks every checkpoint the run wrote.

Which checkpointer. `MemorySaver` is the default here and `SqliteSaver` is opt-in behind `--sqlite PATH`, for three reasons in descending order of importance. `SqliteSaver` lives in a **separate package**, and defaulting to it would inflate the exact dependency bill this lesson is measuring - Lesson 7 set that precedent by keeping `langchain-ollama` opt-in for the same reason. `demo` and `test` must not write to your working tree. And durability still needs demonstrating, so:

```
pip install langgraph-checkpoint-sqlite
./run -l 8 chat --sqlite /tmp/aurora.db --thread aurora "What is the factory reset procedure?"
./run -l 8 chat --sqlite /tmp/aurora.db --thread aurora "Which endpoint exports the logging buffer?"
```

Run those as two separate commands and the turn count keeps going up across two processes. Drop `--sqlite` and the second command starts from one, because `MemorySaver` dies with the process.

In production this is the decision that needs the most thought and usually gets the least. A checkpoint is a verbatim copy of your users' questions and your documents' text, in a database, with a retention policy you have not written yet.

Concept 5 · `interrupt()` - stopping to ask a human

```
def review_node(state):
   decision = interrupt({
       "question": state["question"],
       "query_used": state["query"],
       "attempts": state["attempt"],
       "citations": state["sources"],
       "evidence": [d["text"][:200] for d in state["docs"]],
       "ask": "approve | veto | edit:<new query>",
   })
```

The question the demo pauses on is not chosen at random. `manual.md` says a factory reset **"erases stored credentials, calibration offsets, and the logging buffer"** and warns you to export the buffer first, **"because a factory reset destroys it permanently"**. `warranty.md` says claims arriving without that export **"cannot be assessed and are returned unprocessed"**.

So **"How do I wipe the device and start over?"** is a question where **answering promptly and helpfully costs the reader their evidence and voids their warranty claim**. That is what a human-in-the-loop gate is for. Not a demo checkbox - an irreversible action described in the source documents.

```
  invoke #1 ->  the graph STOPPED. It did not answer.
                status              paused
                get_state().next    ('review',)
                answer in state?    False
                review packet       citations=['manual.md:1', 'troubleshooting.md:1']
                retrievals so far   2

 invoke #2 ->  Command(resume='approve') on the same thread_id
                retrievals now      2      <- unchanged. It resumed; it did not restart.
                status              answered
```

That unchanged integer is the whole proof. The graph did not re-run the question from the top; it picked up at the node it stopped in. One printed number, and the checkpoint stops being an abstract claim.

Three decisions, and the third is why `review` deserves to be a node rather than a flag on `generate`:

- `approve` → `generate`
- `veto` → the answer is never produced. A spy on the provider in the test suite records **zero** calls, which is a different thing from generating an answer and hiding it
- `edit:<a better query>` → back to `retrieve` with the human's words. The reviewer steered the agent, and the run continues rather than ending

You will meet the older static form, `interrupt_before=["generate"]`, in tutorials. It pauses without a payload. Prefer the dynamic form here for a reason that is not stylistic: **the payload is the review packet**, and a human approving a grounded answer needs to see what it is grounded in.

Concept 6 · The grader, and why the default is the worse one

The whole graph turns on one judgement: **is this evidence good enough?** This lesson ships two graders behind one interface, and is opinionated about which you should use.

The deterministic one - term coverage. What fraction of the query's content words appear anywhere in the retrieved text? It is a genuine IR signal, it detects exactly the failure mode this lesson is about, and the terms it reports as **missing** are precisely what the rewriter needs next - so the two nodes are coupled through data rather than a shared hard-coded table.

The LLM one - `--llm-grade`. One model call per grade, a strict brief, lenient parsing, and a loud fallback to coverage that stamps `llm:claude(fell back)` into the trace rather than quietly changing what the loop decides.

The deterministic grader is the default for exactly one reason, and it is not that it is better. `./run -l 8 demo` is committed to `expected-output.txt` and byte-diffed by the test. A lesson whose printed output changes between runs cannot make that promise. That is a property this **lesson** needs. It is almost certainly not a property your system needs.

On your own documents, use `--llm-grade`. It is the better grader. It generalises to phrasings nobody wrote into a glossary, which is the thing the deterministic arm structurally cannot do:

```
./run -l 8 demo  --llm-grade
./run -l 8 ask   "Why is the light orange?" --llm-grade
./run -l 8 trace "What paperwork does an RMA need?" --llm --provider ollama
```

`--llm-grade` swaps the grader, `--llm-rewrite` swaps the rewriter, `--llm` does both, and `--provider` picks between `claude`, `ollama`, `gemini` and `openai`.

Watching it disagree with itself

```
./run -l 8 spread "Is 5GHz supported?" --runs 7
```

This calls the LLM grader seven times against **byte-identical evidence** and prints what came back:

```
 GROUNDED  ###....  3/7
WEAK      ####...  4/7

distinct reasons given
  2x  evidence covers 2.4GHz radios but never names 5GHz
  2x  api.md is unrelated; the band question is unanswered
  ...

the deterministic grader, run 7 times:  WEAK 7/7   (coverage 0.50, missing ['5ghz'])
```

Both verdicts are defensible. `networking.md` does answer the question - by saying the radio **"does not scan the five gigahertz band at all"** - so GROUNDED is right; and the retrieved chunk never contains the string the user typed, so WEAK is right too. **The spread is a property of the question, not a defect of the tool.** A deterministic grader does not remove that ambiguity; it just picks one side of it every time and stops telling you the ambiguity exists.

What to do about it in production, since "use a model and hope" is not advice:

- **Log every `Grade`, including its `reason`.** The reason is what turns a spread into a diagnosis.
- **Alert on the rate, not the instance.** One WEAK on a question that usually passes is noise; a rising weak-rate on a stable corpus means something changed.
- **Pin the grader's prompt like you pin a schema**, and version it. Changing that prompt changes every downstream decision the graph makes.
- **Put both graders in Lesson 5's golden set** and let the evaluation gate tell you whether the LLM grader is worth its calls on your documents.

Concept 7 · Watching it run - LangSmith, and what to use instead

This is the longest section in the lesson, and it is here rather than in Lesson 7 for a specific reason. Lesson 7 noted that a framework puts its dispatch machinery between you and a stack trace. A graph makes that worse in a particular way: **the control flow is now data.** A traceback can tell you where an exception was raised. It cannot tell you why the agent looped three times, which rewrite it chose, which branch `route_after_grade` took, or where it was standing when it paused. The moment you add cycles and interrupts you need a **trace**, not a traceback.

You have already written a tracer

```
trace: Annotated[List[str], operator.add]
```

That is one line, and it is why `./run -l 8 trace` can print this:

```
  retrieve  attempt=1 query='Is 5GHz supported?' -> ['api.md:1', 'networking.md:1']
grade     weak score=0.5 - evidence covers 1/2 query terms; missing ['5ghz']
rewrite   'Is 5GHz supported?' -> 'supported five gigahertz band'  (missing ['5ghz'])
retrieve  attempt=2 query='supported five gigahertz band' -> ['networking.md:1', 'faq.md:1', 'api.md:1']
grade     grounded score=1.0 - evidence covers 4/4 query terms
```

A real tracing tool adds five things to that: **per-node spans with timings**, **token and cost counts**, **inputs and outputs captured at every step**, **run-to-run diffing**, and **replay of a failed run**. All five are worth having. None of them require the tool most tutorials reach for.

What a trace actually is

A trace is a tree of **spans**. Each span has a name, a start and end time, a parent, and a bag of attributes. A linear chain produces a boring tree. A cyclic graph produces one where the same node name appears more than once at different depths, which is exactly the shape you need to see:

```
 run  "Is 5GHz supported?"                                        412 ms
├── retrieve            attempt=1  k=3                            18 ms
│     └── attrs: query="Is 5GHz supported?"  hits=2
├── grade               verdict=weak  score=0.50                   2 ms
│     └── attrs: missing=["5ghz"]  grader="coverage"
├── rewrite             glossary                                   1 ms
│     └── attrs: from="Is 5GHz supported?"  to="supported five gigahertz band"
├── retrieve            attempt=2  k=3           <- the SAME node, again
│     └── attrs: hits=3                                           16 ms
```

```
  ├── grade              verdict=grounded  score=1.00              2 ms
  ├── review             ⏸ interrupted                          373 ms   <- waiting on a human
  └── generate           sources=["networking.md:1"]
```

Two things fall out of that picture that no log line gives you. The repeated `retrieve` is the cycle, visible as structure. And the 373 ms against `review` is not compute - it is a human thinking, which is why **latency percentiles on a graph with an interrupt are meaningless unless you exclude the paused spans.** That trips up nearly everyone the first time.

LangSmith, stated plainly

LangSmith is LangChain's hosted tracing, evaluation and prompt-management platform, and it is what LangGraph integrates with by default. Turning it on is environment variables, not code:

```
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY=...
./run -l 8 trace "The light is orange and it will not connect."
```

Now here is the part worth knowing, which you can verify on your own machine right now:

```
./run -l 8 measure
```

```
  langgraph's dependency closure: ... langchain-core, langsmith, orjson, ...

 Note langsmith in that list. LangChain's hosted tracing client installs
 as a transitive dependency whether you asked for it or not. It is inert
 until you set LANGSMITH_TRACING.
 tracing_is_enabled() right now: False
```

The LangSmith client is already on your machine. It arrived with `langgraph`, via `langchain-core`. It is genuinely dormant - `tracing_is_enabled()` returns `False`, nothing is sent - but it is one environment variable away from being live, and that variable is frequently set in a `.env` someone else wrote.

Exactly what leaves the machine

"Data is sent to a third party" is too vague to act on. For **this** graph, on **these** documents, a hosted tracer records:

Field	What it contains	Sensitivity
`question`	the user's words, verbatim	user data
`query`	the rewritten search query	derived, but still theirs
`docs[].text`	**the retrieved chunks, verbatim**	your documents
rendered prompt	system prompt + the same chunks again	your documents, twice
model output	the generated answer	derived

`Grade.reason`	the grader's justification, quoting evidence	your documents, in fragments
`thread_id`	whatever you chose - often a user or session id	identity

The third row is the one people miss. Tracing a RAG system does not export metadata about your documents; **it exports your documents**, a chunk at a time, one span per retrieval. For a course called `local-ai-lab`, running on a corpus you deliberately kept on your own disk, that deserves to be a decision rather than a default.

For completeness: LangSmith can be self-hosted, but that is an Enterprise-licensed product, not the free tier. Worth knowing before you plan around it.

Where LangSmith genuinely earns its place: a team that wants annotation queues, prompt versioning, and dataset-backed evaluations in one product, without operating anything. That is a real need and it is well served. It is simply not the only option, and it is not the default this course would pick.

The alternatives, in detail

Tool	Install	Where data lives	Licence	Best at
Phoenix (Arize)	`pip install arize-phoenix`	**your machine**	open source	local-first RAG tracing and eval; OTel-native; runs from a terminal or a notebook
Langfuse	Docker compose, or hosted	your infra, or theirs	OSS core, some features paid	the closest drop-in to LangSmith: tracing, evals, prompt management
OpenLLMetry (Traceloop)	`pip install traceloop-sdk`	wherever your OTel backend is	open source	putting LLM spans into Jaeger, Tempo or Honeycomb - the stack you already run
OpenTelemetry GenAI conventions	already in your stack	anywhere	open standard	instrumenting once and staying portable
MLflow Tracing	`pip install mlflow`	your machine or tracking server	open source	teams already running MLflow
Opik (Comet)	self-host or hosted	your infra, or theirs	open source	tracing plus eval with hosted-style ergonomics

Helicone	proxy, self-hostable	your infra, or theirs	open source	a drop-in proxy when you cannot change app code

A word on "open source" in this space: several of these are open-core, where self-hosting gets you the community edition and some features stay behind a licence. That is a legitimate model, but check which tier has the feature you are planning around **before** you build on it.

The trace/eval boundary

Readers routinely conflate these, and the tools do not help by selling both.

- **A trace tells you what happened.** Which nodes ran, in what order, how long, with what inputs.
- **An eval tells you whether it was any good.** Scored against expected behaviour, over a set of questions, tracked across versions.

This course already built the second one, in Lesson 5. The `spread` command in Concept 6 is a hand-rolled eval - it runs one grader many times and reports a distribution. Where that belongs permanently is Lesson 5's golden set. For scored RAG metrics specifically - faithfulness, answer relevancy, context precision - **Ragas** maps almost directly onto what Lesson 5 built, and `promptfoo` and `DeepEval` cover prompt regression testing. None of those are tracing tools, and reaching for a trace viewer to answer "is this any good" is the most common wrong turn here.

The recommendation

Instrument with OpenTelemetry; look at it in Phoenix, locally. That keeps the backend a deployment decision rather than a code dependency, and it keeps a course about local, private AI actually local and private. If your team later wants Langfuse or LangSmith, the instrumentation does not change - only where you point it.

Roughly ten lines, and none of them are in this lesson's `requirements.txt` on purpose, so the dependency scorecard stays honest:

```
import phoenix as px
from openinference.instrumentation.langchain import LangChainInstrumentor
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import SimpleSpanProcessor
from opentelemetry.exporter.otlp.proto.http.trace_exporter import OTLPSpanExporter

px.launch_app()                                  # a local UI, no account
provider = TracerProvider()
provider.add_span_processor(SimpleSpanProcessor(OTLPSpanExporter()))
LangChainInstrumentor().instrument(tracer_provider=provider)
# now run the graph exactly as before - every node becomes a span
```

In production

- **Sample.** One question through this graph emits five to nine spans, and a corrective loop emits more than a chain by design. Tracing every request at full fidelity is a bill.
- **Redact before export, not after.** The retrieved-chunk attribute is the one to filter. Once it has left the process, "we'll clean it up later" is not a plan.

- **Match retention to the checkpointer's.** Traces and checkpoints hold the same text. Two different retention policies on the same data is one policy that does not work.
- **Make `thread_id` the join key** between a trace and a checkpoint, so a support question about one run can be answered by looking at both.

When a graph beats a chain

Dimension	LangGraph wins	A `while` loop wins
Answer quality	nothing - identical on all 9	nothing - identical on all 9
Lines of code	170 lines of wiring	**61 lines**
Dependencies	+23 packages (+5 after Lesson 7)	**zero**
Debuggability	a trace, and a queryable topology	a stack trace that is your own code
State across turns	**checkpointer +** `thread_id`	you write it, and its migrations
Pause and resume	`interrupt()` / `Command(resume=)`	you cannot, without restructuring
Introspection	`get_graph().nodes` / `.edges`	read the source
Fan-out / parallel nodes	**built in**	you write it
Onboarding a new engineer	a diagram that is generated, not drawn	a function they can read in one sitting

The `while` loop matched it on every answer. Everything you paid for is in the other rows.

Do not reach for a graph when you retrieve once, no human is ever in the path, and nothing needs to be remembered between turns - you will have bought a state machine you never checkpoint, and its dispatch machinery will sit between you and every stack trace for no return. `CHEATSHEET.md` puts it in one line: **linear pipelines a plain function handles** is the "don't" column.

Do reach for it when any of the bottom four rows is a real requirement - and for anything touching an irreversible action, the pause row is not optional. There is no cheap way to write resumable, inspectable, human-gated control flow yourself, and every team that tries writes a worse LangGraph.

Where did tool routing go? The roadmap outline for this lesson promised it. It moved to **Lesson 9 · Ollama + function calling**, which is the tool-calling lesson, and where letting a **model** choose which tool to call can be taught on its own terms instead of as a footnote to control flow. `CHEATSHEET.md` already draws that line. This lesson is about deciding **whether to run a step again**; Lesson 9 is about deciding **which step to run**.

Python and Node

Both official SDKs are here, and the retrieval and control flow are **identical** - BM25 and the coverage grader are deterministic string arithmetic, so both runtimes reach the same documents in the same order and take the same branches:

```
  The graph agrees with the while loop on 9/9 questions.
Same answers, same attempts, same retrievals - in a second runtime.
```

The bill deliberately is not identical:

Runtime	Packages	Install size	Notes
Python (`langgraph`)	+23, to 71	~43 MB	+5 only if you already did Lesson 7
Node (`@langchain/langgraph`)	+23, to 23	~63 MB	from nothing at all; needs Node 22

One genuine API difference, worth knowing before you port anything: **the JavaScript `StateGraph` refuses to let a state channel share a name with a node.** The Python state has a `grade` field and a `grade` node; the Node port had to rename the field to `verdict`. The Python SDK allows it. Neither is wrong, but a line-for-line port will not run.

The Node port stops at `demo`, exactly as Lesson 7's did, because the provider stack and the playground are Python. Cross-language parity lives in the byte-checked comparison, not in a second copy of the GUI.

There is no C# port because there is no official LangGraph for .NET. **.NET is not being skipped** - Microsoft's answer to this whole problem is **Semantic Kernel**, and it gets Lesson 10 to itself.

Exercises

- **Break the grader:** find a question where coverage says `grounded` and the retrieval is plainly wrong. (Start with questions whose words all appear in the corpus but in unrelated documents.) Then decide: is the fix a higher threshold, or a better grader? Run it again with `--llm-grade` and see whether the model catches what the arithmetic missed.
- **Delete the glossary:** empty `data/glossary.json` and re-run the demo. How many of the four fixes survive? Now run `--llm-rewrite` and count again. That difference is the exact price of having a reproducible lesson, and the exact reason you should not pay it in production.
- **Add a node, then watch it:** insert a `dedupe` node between `retrieve` and `grade` that drops chunks from a source already cited this run - two lines of wiring and one function. Did it change any attempt counts? Then `pip install arize-phoenix`, instrument with the ten lines from Concept 7, and re-run `q4`: your new node is a new span. How many spans does one corrective loop emit, and would you sample that in production?

- **Push it onto your own project:** point `CORPUS_DIR` at your own documents, write ten questions your team actually asks, label the `expect` column by hand, and run all three arms. Report one number: **how many of the ten does the linear chain get wrong, and how many does the loop fix?** If the answer is zero, you have just saved yourself a dependency - and that is a real result, not a failure.

From demo to production

- **Pin `langgraph` exactly.** The interrupt API moved recently; `Command(resume=)` is not what a tutorial from a year ago will show you. Read the changelog before every bump.
- **Give the checkpointer real storage and a retention policy.** A checkpoint is a verbatim copy of your users' questions and your documents' text. Decide how long that lives before you write the first one.
- **Make `thread_id` a real identity** - a conversation, a ticket, a session - and use the same value as your trace's join key. A hash of the query means two people asking the same thing share memory.
- **Set both caps, and alert on the abstain rate.** One refusal is correct behaviour. A rising refusal rate on a stable corpus is a retrieval problem announcing itself early, which is the most useful signal this whole design produces.
- **Gate the irreversible things, not everything.** An interrupt on every answer is a queue nobody reads. An interrupt before an action that destroys data is the reason this feature exists.
- **Instrument with OpenTelemetry and redact the retrieved-chunk attribute** before it leaves the process. See Concept 7 for what is actually in a span.
- **Fold it into Lesson 5.** Put all three arms in the golden set and let the evaluation gate tell you whether the loop helps on **your** documents, instead of trusting nine questions someone else wrote.
- **Keep the citation contract at the boundary.** `source:page` has now survived a framework rewrite **and** a control-flow rewrite untouched. Make that the thing your tests assert, and you can change how the answer is produced without changing what you promise callers.

Next lesson

Lesson 9 · Ollama + Function Calling → - stop routing tools yourself and let a local model decide when to call them, fully offline.
